

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
Belgaum, Karnataka



DATABASE LABORATORY
LABORATORY MANUAL
MMCL206

II SEMESTER - 2024

Prepared By:
Dr. Rajani Narayan
Associate Professor
Dept. of MCA, RNSIT

Department of MCA



ESTD: 2001

An Institution with a difference

Program 1

Create the following tables with properly specifying Primary keys, Foreign keys and solve the following queries.

BRANCH(Branchid, Branchname, HOD)

STUDENT(USN, Name, Address, Branchid, sem)

BOOK(Bookid, Bookname, Authorid, Publisher, Branchid)

AUTHOR(Authorid, Authurname, Country, age)

BORROW(USN, Bookid, Borrowed_Date)

Queries:

- 1 List the details of Students who are all Studying in 2nd sem MCA.
- 2 List the students who are not borrowed any books.
- 3 Display the USN, Student name, Branch_name, Book_name, Author_name , Books_Borrowed_Date of 2nd sem MCA Students who borrowed books.
- 4 Display the number of books written by each Author.
- 5 Display the student details who borrowed more than two books.
- 6 Display the student details who borrowed books of more than one Author.
- 7 Display the Book names in descending order of their names.
- 8 List the details of students who borrowed the books which are all published by the same Publisher.

BRANCH

<u>Branchid</u>	<u>Branchname</u>	HOD
-----------------	-------------------	-----

STUDENT

<u>USN</u>	Name	Address	Branchid	Sem
------------	------	---------	----------	-----

BOOK

<u>Bookid</u>	Bookname	Authorid	Publisher	Branchid
---------------	----------	----------	-----------	----------

AUTHOR

<u>Authorid</u>	Authurname	Country	Age
-----------------	------------	---------	-----

BORROW

<u>USN</u>	<u>Bookid</u>	Borrowed_Date
------------	---------------	---------------

ORACLE 10g SQL Installation

http://www.mediafire.com/file/x96d2c02t8x45xa/Oracle_Database_10G_Enterprise_edition_32_bit.zip/file

Create all the 5 tables

**1. CREATE TABLE BRANCH
(BRANCHID INT PRIMARY KEY, BNAME VARCHAR(10),
HOD VARCHAR(10));**

```
INSERT INTO BRANCH VALUES (1,'MCA','N P Kavya');
INSERT INTO BRANCH VALUES (2,'BE_CS','Kiran');
INSERT INTO BRANCH VALUES (3,'BE_EC','VipulSing');
INSERT INTO BRANCH VALUES (4,'BE_ISE','L Suresh');
INSERT INTO BRANCH VALUES (5,'MATHS','Suresh');
INSERT INTO BRANCH VALUES (6,'BE_PHY','Panduran');
```

```
SQL> select * from branch;
```

BRANCHID	BNAME	HOD
1	MCA	N P KAVYA
2	BE_CS	Kiran
3	BE_EC	Vipul Singh
4	ISE	L Suresh
5	Maths	Suresh

**2. CREATE TABLE STUDENT
(USN VARCHAR(10) PRIMARY KEY, NAME VARCHAR(30), ADDR
VARCHAR(15), BRANCHID INT REFERENCES BRANCH(BRANCHID),
SEM INT);**

```
INSERT INTO STUDENT VALUES ( '1RN1', 'harish', 'bengalore', 1, 2);
INSERT INTO STUDENT VALUES ( '1RN2', 'bharath', 'mysore', 2, 3);
INSERT INTO STUDENT VALUES ( '1RN3', 'kiran', 'delhi', 3, 6);
INSERT INTO STUDENT VALUES ( '1RN4', 'mahi', 'Chennai', 4, 7);
INSERT INTO STUDENT VALUES ( '1RN5', 'Krishna', 'hubli', 5, 4);
INSERT INTO STUDENT VALUES ( '1RN6', 'Krish', 'Ranchi', 6, 2);
```

```
SQL> select * from student;
```

USN	NAME	ADDR	BRANCHID	SEM
1rn1	harish	bangalore	1	2
1rn2	bharath	mysore	2	3
1rn3	kiran	delhi	3	6
1rn4	mahi	chennai	4	7
1rn5	krishna	hubli	5	4

3. CREATE TABLE AUTHOR
(AUTHORID INT PRIMARY KEY,
ANAME VARCHAR(10),
COUNTRY VARCHAR(10),
AGE INT);

INSERT INTO AUTHOR VALUES (123, 'navathe', 'india', 55);
 INSERT INTO AUTHOR VALUES (124, 'ritche', 'uk', 44);
 INSERT INTO AUTHOR VALUES (125, 'Ramakrishn', 'india', 55);
 INSERT INTO AUTHOR VALUES (126, 'sumitabha', 'india', 38);
 INSERT INTO AUTHOR VALUES (127, 'dennis', 'usa', 66);

SQL> select * from author;

AUTHORID	ANAME	COUNTRY	AGE
123	navathe	india	55
124	ritche	uk	44
125	RAMKRISHNA	india	55
126	sumitabha	india	38
127	dennis	usa	66

4. CREATE TABLE BOOK
(BOOKID INT PRIMARY KEY,
BNAME VARCHAR(10),
AUTHORID INT REFERENCES AUTHOR(AUTHORID),
PUBLISHER VARCHAR(10),
BRANCHID INT REFERENCES BRANCH(BRANCHID));

INSERT INTO BOOK VALUES (1111, 'c prog', 123, 'pearson', 1);
 INSERT INTO BOOK VALUES (2222, 'dbms', 124, 'mrawhill', 2);
 INSERT INTO BOOK VALUES (3333, 'oops', 125, 'sapna', 3);
 INSERT INTO BOOK VALUES (4444, 'unix', 126, 'subhash', 4);
 INSERT INTO BOOK VALUES (5555, 'cprog', 127, 'pearsonh', 5);

SQL> select * from book;

BOOKID	BNAME	AUTHORID	PUBLISHER	BRANCHID
1111	c prog	123	pearson	1

2222 dbms	124 mdrawhill	2
3333 oops	125 sapna	3
4444 unix	126 subhash	4
5555 cprog	127 pearson	5

```
5. CREATE TABLE BORROW
(USN VARCHAR(10) REFERENCES STUDENT(USN),
BOOKID INT REFERENCES BOOK(BOOKID),
BORROWDATE DATE);
```

```
INSERT INTO BORROW VALUES( '1RN1',2222,'10-JAN-00');
INSERT INTO BORROW VALUES( '1RN1',3333, '05-MAR-16');
INSERT INTO BORROW VALUES( '1RN3',5555, '01-JUN-10');
INSERT INTO BORROW VALUES( '1RN5',2222, '19-MAY-00');
INSERT INTO BORROW VALUES( '1RN2',1111, '22-FEB-15');
```

```
SQL> select * from borrow;
```

USN	BOOKID	BORROWDAT
1RN1	2222	10 - JAN - 00
1RN1	3333	05 - MAR - 16
1RN3	5555	01 - JUN - 10
1RN5	2222	19 - MAY - 00
1RN2	1111	22 - FEB - 15

1. List the details of Students who are all Studying in 2nd sem MCA

Query 1.

```
select * from student where sem=2 and branchid in
(select branchid from branch where bname='MCA');
```

USN	NAME	ADDR	BRANCHID	SEM
1RN1	harish	bangalore	1	2

2. List the students who are not borrowed any books.

Query 2.

```
select * from student where usn not in (select usn from borrow);
```

USN	NAME	ADDR	BRANCHID	SEM
1RN4	mahi	chennai	4	7

3.Display the USN, Student name, Branch_name, Book_name, Author_name , Books_Borrowed_Date of 2nd sem MCA Students who borrowed books.

Query 3.

```
select student.usn ,student.name,branch.bname, book.bname, aname,
borrowdate from student , branch, book, author, borrow where
student.usn=borrow.usn and borrow.bookid=book.bookid and book.authorid
=author.authorid and student.branchid=branch.branchid and student.sem=2
and branch.bname='MCA';
```

USN	NAME	BNAME	BNAME	ANAME	BORROWDAT
1RN1	harish	MCA	dbms	ritche	10-JAN-00
1RN1	harish	MCA	oops	RAMKRISHN	05-MAR-16

4. Display the number of books written by each Author.

Query 4.

```
select count(*) , authorid from book group by authorid;
```

COUNT(*)	AUTHORID
1	123
1	125
1	124
1	126
1	127

5. Display the student details who borrowed more than two books.

Query 5.

```
select * from student where usn in ( select usn from borrow group by
usn having count(usn) >=2);
```

USN	NAME	ADDR	BRANCHID	SEM
1RN1	harish	bangalore	1	2

6.Display the student details who borrowed books of more than one Author.

Query 6.

```
select * from student s where exists (select br.usn from borrow br join
book bk on br.bookid=bk.bookid where br.usn=s.usn group by usn having
count(distinct authorid)>1);
```

USN	NAME	ADDR	BRANCHID	SEM
1RN1	harish	bangalore	1	2

7. Display the Book names in descending order of their names.

Query 7.

```
select bname from book order by bname desc;
```

```
BNAME
-----
unix
Oops
Dbms
cprog
c prog
```

8. List the details of students who borrowed the books which are all published by the same Publisher.

Query 8.

```
select * from student s where exists (select usn , publisher from
borrow join book on borrow.bookid=book.bookid where
s.usn=borrow.usn group by usn having count(distinct publisher)=1);
```

USN	NAME	ADDR	BRANCHID	SEM
1rn2	bharath	mysore	2	3
1rn3	kiran	delhi	3	6
1rn5	krishna	hubli	5	4

2. Create the following schema.

STUDENT(USN,name,date_of_birth,branch,mark1,mark2,mark3,total,GPA)

Execute the following queries

- i) Update the column total by adding the column mark1,mark2,mark3.
- ii) Find the GPA score of all the students.
- iii) Find the students who born on a particular year of birth from the date_of_birth column.
- iv) List the students who are studying in a particular branch of study.
- v) Find the maximum GPA score of the student branch-wise.
- vi) Find the students whose name starts with alphabet "S".
- vii) Find the students whose name ends with alphapets "AR".
- viii) Delete the student details whose USN is given as 1001.

create table StudentR(USN varchar(10) primary key, Sname varchar(20), Date_of_Birth Date, Branch varchar(10), Mark1 int, Mark2 int, Mark3 int, Total int, GPA int);

Table created.

SQL> insert into StudentR values('1RN23MC047','Pavan','15-jul-2002','MCA',91,82,86,0,0);
1 row created.

SQL> insert into StudentR values('1RN23MC045','Kushal','3-sep-2000','MCA',91,82,86,0,0);
1 row created.

SQL> select * from StudentR;

USN	SNAME	DATE_OF_B	BRANCH	MARK1	MARK2	MARK3	TOTAL	GPA
1RN23MC047	Pavan	15-JUL-02	MCA	91	82	86	0	0
1RN23MC045	Kushal	03-SEP-00	MCA	91	82	86	0	0

SQL> update StudentR set total = Mark1+Mark2+Mark3;
2 rows updated.

SQL> set linesize 150 pagesize 100;

SQL> select * from StudentR;

USN	SNAME	DATE_OF_B	BRANCH	MARK1	MARK2	MARK3	TOTAL	GPA
1RN23MC047	Pavan	15-JUL-02	MCA	91	82	86	259	0
1RN23MC045	Kushal	03-SEP-00	MCA	91	82	86	259	0

SQL> update StudentR set GPA = total/3;

2 rows updated.

SQL> select * from StudentR where Date_of_Birth like '%02';

USN	SNAME	DATE_OF_B	BRANCH	MARK1	MARK2	MARK3	TOTAL	GPA
1RN23MC047	Pavan	15-JUL-02	MCA	91	82	86	259	86

SQL> select * from StudentR where Date_of_Birth like '15%';

USN	SNAME	DATE_OF_B	BRANCH	MARK1	MARK2	MARK3	TOTAL	GPA
1RN23MC047	Pavan	15-JUL-02	MCA	91	82	86	259	86

SQL> select * from StudentR Where Branch = 'CS';

no rows selected

SQL> select * from StudentR Where Branch = 'MCA';

USN	SNAME	DATE_OF_B	BRANCH	MARK1	MARK2	MARK3	TOTAL	GPA
1RN23MC047	Pavan	15-JUL-02	MCA	91	82	86	259	86
1RN23MC045	Kushal	03-SEP-00	MCA	91	82	86	259	86

SQL> select branch , max(GPA) from StudentR group by Branch;

BRANCH	MAX(GPA)
MCA	86

SQL> select * from StudentR where Sname like '%an';

USN	SNAME	DATE_OF_B	BRANCH	MARK1	MARK2	MARK3	TOTAL	GPA
1RN23MC047	Pavan	15-JUL-02	MCA	91	82	86	259	86

SQL> select * from StudentR where Sname like 'p%';

no rows selected

SQL> select * from StudentR where Sname like 'P%';

USN	SNAME	DATE_OF_B	BRANCH	MARK1	MARK2	MARK3	TOTAL	GPA
1RN23MC047	Pavan	15-JUL-02	MCA	91	82	86	259	86

SQL> select * from StudentR where Sname like 'K%';

USN	SNAME	DATE_OF_B	BRANCH	MARK1	MARK2	MARK3	TOTAL	GPA
1RN23MC045	Kushal	03-SEP-00	MCA	91	82	86	259	86

SQL> delete from StudentR Where USN like '1RN23MC047';

1 row deleted.

SQL> select * from StudentR;

USN	SNAME	DATE_OF_B	BRANCH	MARK1	MARK2	MARK3	TOTAL	GPA
-----	-------	-----------	--------	-------	-------	-------	-------	-----

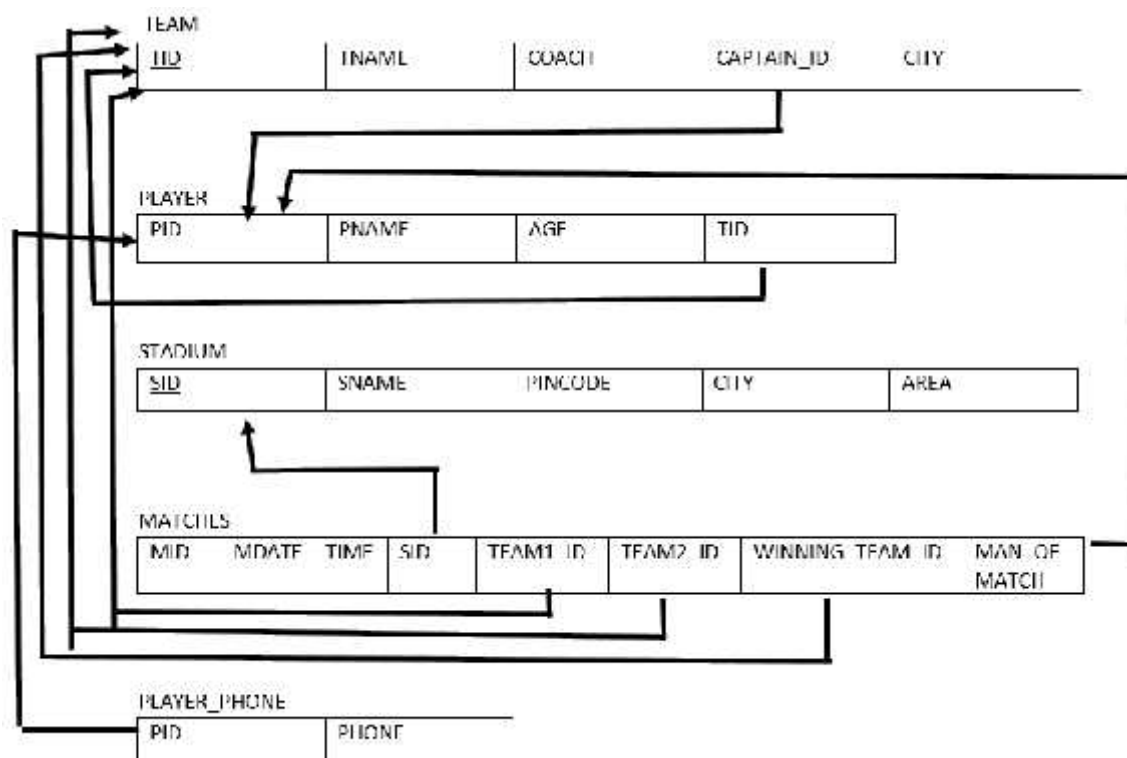
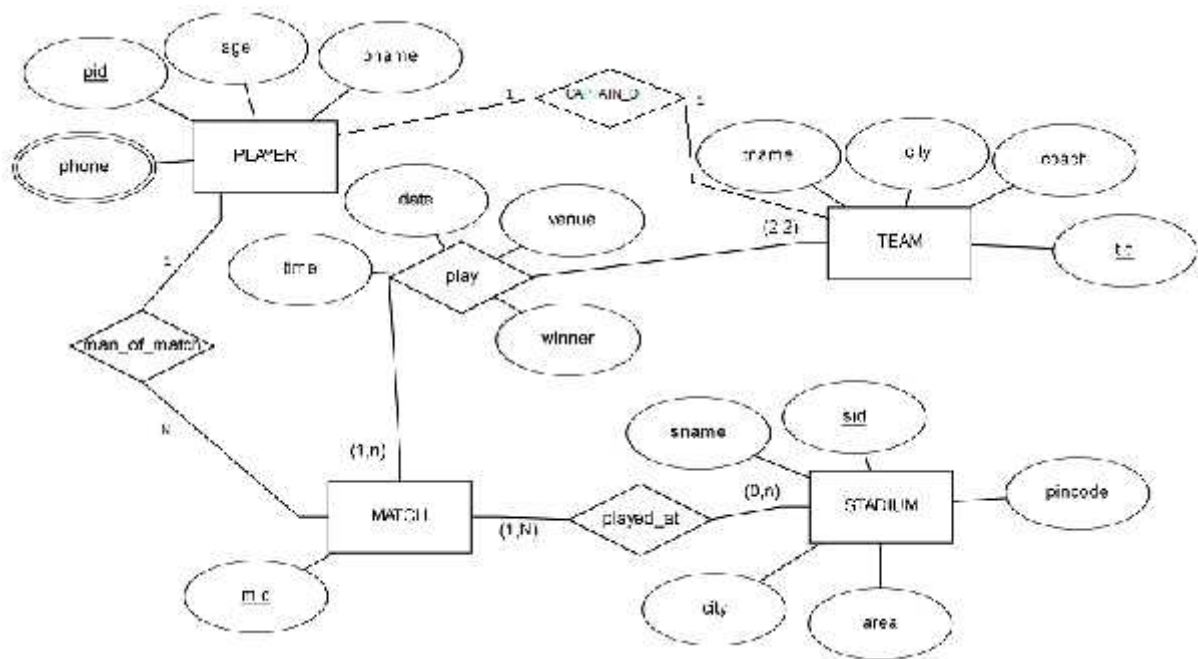
-----	-----	-----	-----	-----	-----	-----	-----	-----
1RN23MC045	Kushal	03-SEP-00	MCA	91	82	86	259	86

3. Design an ER-diagram for the following scenario, Convert the same into a relational model and then solve the following queries.

Consider a Cricket Tournament “ABC CUP” organized by an organization. In the tournament there are many teams are contesting each having a Teamid, Team_Name, City, a coach. Each team is uniquely identified by using Teamid. A team can have many Players and a captain. Each player is uniquely identified by Playerid, having a Name, and multiple phone numbers, age. A player represents only one team. There are many Stadiums to conduct matches. Each stadium is identified using Stadiumid, having a stadium_name, Address (involves city, area_name, pincode). A team can play many matches. Each match played between the two teams in the scheduled date and time in the predefined Stadium. Each match is identified uniquely by using Matchid. Each match won by any of the one team that also wants to record in the database. For each match man_of_the match award given to a player.

Queries:

1. Display the youngest player (in terms of age) Name, Team name , age in which he belongs of the tournament.
2. List the details of the stadium where the maximum number of matches were played.
3. List the details of the player who is not a captain but got the man_of _match award at least in two matches.
4. Display the Team details who won the maximum matches.
5. Display the team name where all its won matches played in the same stadium.



I.

create table team (tid int primary key, tname varchar(20), coach varchar(20), captain_pid int, city varchar(20));

```

INSERT INTO TEAM VALUES(123,'rcb','sunil',1,'bangalore');
INSERT INTO TEAM VALUES(124,'csk','laxman',3,'chennai');
INSERT INTO TEAM VALUES(125,'royals','singh',4,'rajasthan');
INSERT INTO TEAM VALUES(126,'daredevils','sehwag',2,'delhi');
INSERT INTO TEAM VALUES(127,'kkr','ravi',5,'goa');
  
```

```
SQL> select * from team;
```

TID	TNAME	COACH	CAPTAIN_PID	CITY
123	rcb	sunil	1	bangalore
124	csk	laxman	3	chennai
125	royals	singh	4	rajasthan
126	daredevils	sehwag	2	delhi

II.

```
create table player (pid int primary key, pname varchar(20),
age int, tid int references team(tid));
```

```
insert into player values(1, 'sachin',33,123);
insert into player values(2, 'dravid',32,124);
insert into player values(3, 'dhoni',30,124);
insert into player values(4, 'raina',30,125);
insert into player values(5,'kohli',23,126);
```

```
SQL> select * from player;
```

PID	PNAME	AGE	TID
1	sachin	33	123
2	dravid	32	124
3	dhoni	30	124
4	raina	30	125
5	kohli	23	126

III. create table stadium (sid int primary key, sname varchar(20),
picode number(8), city varchar(20), area varchar(20));

```
insert into stadium values(111,'chinnaswamy',56001,'bangalore','mg road');
insert into stadium values(222,'kotla',460009,'delhi','highway');
insert into stadium values(333,'international',38883,'chennai','trnagar');
insert into stadium values(444,'ksca',560098,'bangalore','peenya');
insert into stadium values(555,'csca',567772,'cochin','beach road');
```

```
SQL> select * from stadium;
```

SID	SNAME	PICODE	CITY	AREA
111	chinnaswamy	56001	bangalore	mg road
222	kotla	460009	delhi	highway
333	international	38883	chennai	tr nagar
444	ksca	560098	bangalore	peenya

555 csca

567772 cochin

beach road

IV.

```
create table match (mid int primary key, mdate date,
time varchar(6),
sid int references stadium(sid),
team1_id int references team(tid),
team2_id int references team(tid),
winning_team_id int references team(tid),
man_of_match int references player(pid));
```

```
insert into match values(101,'10-JAN-17','10am',111,123,124,123,1);
insert into match values(102,'11-JAN-17','9am',222,124,126,126,5);
insert into match values(103,'12-JAN-17','11am',111,125,126,126,5);
insert into match values(104,'10-JAN-17','12am',111,125,123,123,1);
insert into match values(10,'11-JAN-17','9am',222,124,126,126,5);
```

SQL> select * from match;

MID	MDATE	TIME	SID	TEAM1_ID	TEAM2_ID	WINNING_TEAM_ID	MAN_OF_MATCH
101	10-JAN-17	10am	111	123	124	123	1
102	11-JAN-17	pm	222	124	126	126	5
103	12-JAN-17	11am	111	125	126	126	5
104	17-JAN-17	12pm	111	125	123	123	1

v. create table player_phone
(pid int references
player(pid), phone int,
primary
key(pid,phone));

```
insert into player_phone values(1,998882928);
insert into player_phone values(2,877563733);
insert into player_phone values(2,988928822);
insert into player_phone values(2,877366383);
```

SQL> select * from player_phone;

PID	PHONE
1	998882928
2	877563733
2	988928822
3	877366383

Query 1 :

```
Select pname, tname, age from player p, team t where
p.tid=t.tid and age =(select min(age) from player);
```

PNAME	TNAME	AGE
kohli	daredevils	23

Query 2:

```
select * from stadium where sid in
(select sid from match group by sid having count(sid) =
(select max(count(sid)) from match group by sid))
```

SID	SNAME	PICODE	CITY	AREA
111	chinnaswamy	56001	bangalore	mg road

Query 3:

```
select * from player where pid not in ( select captain_pid from
team) and pid in (select man_of_match from match group by
man_of_match having count(man_of_match)=2);
```

PID	PNAME	AGE	TID
5	kohli	23	126

Query 4:

```
select * from team where tid in (select winning_team_id from
match group by winning_team_id having count(winning_team_id)=
(select max(count(winning_team_id))from match group by
winning_team_id))
```

TID	TNAME	COACH	CAPTAIN_PID	CITY
126	daredevils	sehwag	2	delhi

Query 5:

```
select tname from team where tid in (
select winning_team_id from match group
by(winning_team_id,sid)
having count(*) in (select count(winning_team_id)
from match group by winning_team_id));
```

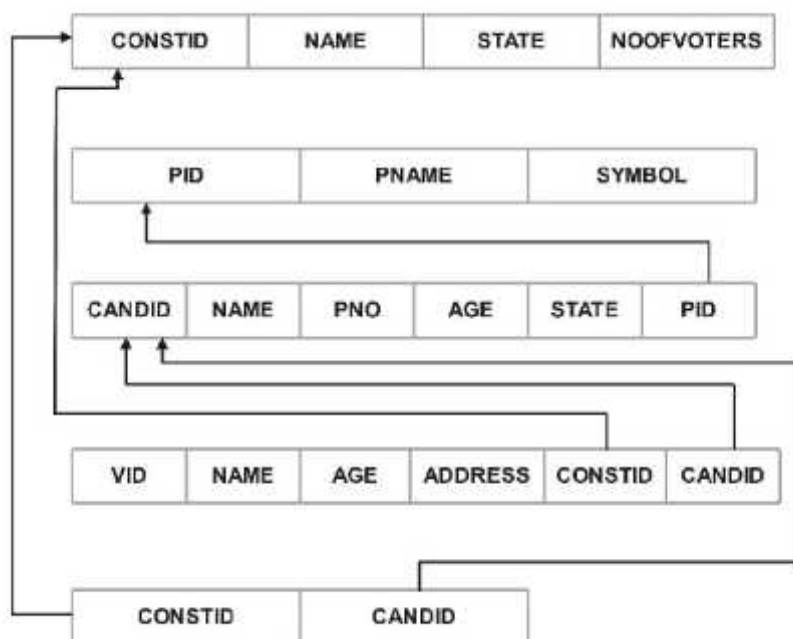
TNAME
rcb

4. A country wants to conduct an election for the parliament. A country having many constituencies. Each constituency is identified uniquely by Constituency_id, having the Name, belongs to a state, Number_of_voters. A constituency can have many voters. Each voter is uniquely identified by using Voter_id, having the Name, age, address (involves Houseno,city,state,pincode). Each voter belongs to only one constituency. There are many candidates contesting in the election. Each candidates are uniquely identified by using candidate_id, having Name, phone_no, age, state. A candidate belongs to only one party. There are many parties. Each party is uniquely identified by using Party_id, having Party_Name, Party_symbol. A candidate can contest from many constituencies under a same party. A party can have many candidates contesting from different constituencies.

No constituency having the candidates from the same party. A constituency can have many contesting candidates belongs to different parties. Each voter votes only one candidate of his/her constituency.

Queries:

- List the details of the candidates who are contesting from more than one constituencies which are belongs to different states.
- Display the state name having maximum number of constituencies.
- Create a stored procedure to insert the tuple into the voter table by checking the voter age. If voter's age is at least 18 years old, then insert the tuple into the voter else display the "Not an eligible voter msg".
- Create a stored procedure to display the number_of_voters in the specified constituency. Where the constituency name is passed as an argument to the stored procedure.
- Create a TRIGGER to UPDATE the count of "Number_of_voters" of the respective constituency in "CONSTITUENCY" table, AFTER inserting a tuple into the "VOTERS" table.




```
CREATE TABLE CONSTITUENCY(
  CONSTID INT PRIMARY KEY,
  NAME VARCHAR(30),
  STATE VARCHAR(30),
  NOOFVOTERS INT
);
```

```
INSERT INTO CONSTITUENCY VALUES(1,'BANGALORE','KARNATAKA',2);
INSERT INTO CONSTITUENCY VALUES(2,'SHIMOGA','KARNATAKA',2);
INSERT INTO CONSTITUENCY VALUES(3,'HYDERABAD','TELANGANA',3);
```

```
CREATE TABLE PARTY(
  PID INT PRIMARY KEY,
  PNAME VARCHAR(30),
  SYMBOL VARCHAR(30)
);
```

```
INSERT INTO PARTY VALUES(1,'CONGRESS','PALM');
INSERT INTO PARTY VALUES(2,'BJP','LOTUS');
```

```
CREATE TABLE CANDIDATE(
  CANDID INT PRIMARY KEY,
  NAME VARCHAR(30),
  PNO VARCHAR(30),
  AGE INT,
  STATE VARCHAR(30),
  PID INT REFERENCES PARTY(PID)
);
```

```
INSERT INTO CANDIDATE VALUES(1,'AMAR','8660639260',35,'KARNATAKA',1);
INSERT INTO CANDIDATE VALUES(2,'ANTONY','8660639262',37,'TELANGANA',2);
```

```
CREATE TABLE VOTERS(
  VID INT,
  NAME VARCHAR(30),
  AGE VARCHAR(30),
  ADDRESS VARCHAR(150),
  CONSTID INT REFERENCES CONSTITUENCY(CONSTID),
  CANDID INT REFERENCES CANDIDATE(CANDID)
);
```

```
INSERT INTO VOTERS VALUES(1,'ABHI',20,'1 BALKI BIDAR KARNATAKA 577001',1,1);
INSERT INTO VOTERS VALUES(2,'ARIF',20,'1 BHADRAVATHI BHADRAVATHI
KARNATAKA
577201',2,2);
INSERT INTO VOTERS VALUES(3,'MANOJ',18,'1 ANAVERI SHIMOGA KARNATAKA
577243',1,2);
```

```
CREATE TABLE CONTEST(
  CONSTID INT REFERENCES CONSTITUENCY(CONSTID),
```

```
CANDID INT REFERENCES CANDIDATE(CANDID)
);
```

```
INSERT INTO CONTEST VALUES(1,1);
INSERT INTO CONTEST VALUES(2,2);
INSERT INTO CONTEST VALUES(3,1);
```

- i. List the details of the candidates who are contesting from more than one constituencies which are belongs to different states.

```
SELECT *
FROM CANDIDATE
WHERE CANDID IN (
  SELECT CANDID
  FROM CONTEST CT, CONSTITUENCY CS
  WHERE CT.CONSTID = CS.CONSTID
  GROUP BY CANDID
  HAVING COUNT(DISTINCT(STATE))>1
);
```

CANDID	NAME	PNO	AGE	STATE
1	AMAR	8660639260	35	KARNATAKA

- ii. Display the state name having maximum number of constituencies.

```
SELECT STATE FROM CONSTITUENCY
GROUP BY STATE
HAVING COUNT(STATE) = (
  SELECT MAX(COUNTS) FROM (
    COUNT(*) COUNTS
    FROM CONSTITUENCY
    GROUP BY STATE) COUNTER
);
```

```
STATE
-----
KARNATAKA
```

- iii. Create a stored procedure to insert the tuple into the voter table by checking the voter age. If voter's age is at least 18 years old, then insert the tuple into the voter else display the "Not an eligible voter msg".

```
CREATE OR REPLACE PROCEDURE CHECKAGE(ID IN NUMBER,AGE IN NUMBER)
AS
BEGIN
  IF AGE>=18 THEN
    INSERT INTO VOTERS(VID,AGE)VALUES(ID,AGE);
  ELSE
    DBMS_OUTPUT.PUT_LINE('NOT AN ELIGIBLE VOTER');
  END IF;
```

END;

Procedure created.

SQL> SET SERVEROUTPUT ON;

SQL> EXEC CHECKAGE(4,17);

NOT AN ELIGIBLE VOTER

PL/SQL procedure successfully completed.

SQL> EXEC CHECKAGE(3,22);

PL/SQL procedure successfully completed.

iv. Create a stored procedure to display the number_of_voters in the specified constituency.

Where the constituency name is passed as an argument to the stored procedure.

CREATE OR REPLACE PROCEDURE COUNTVOTERS(ID IN NUMBER)

AS

CNT CONSTITUENCY.CONSTID % TYPE;

BEGIN

SELECT NOOFVOTERS INTO CNT

FROM CONSTITUENCY

WHERE CONSTID = ID AND ROWNUM = 1;

DBMS_OUTPUT.PUT_LINE ('TOTAL VOTERS ARE: ' || CNT);

END;

Procedure created.

SQL> EXEC COUNTVOTERS(1);

TOTAL VOTERS ARE: 2

PL/SQL procedure successfully completed.

v. Create a TRIGGER to UPDATE the count of "Number_of_voters" of the respective

constituency in "CONSTITUENCY" table , AFTER inserting a tuple into the "VOTERS" table.

CREATE OR REPLACE TRIGGER UPDATECOUNT

AFTER INSERT ON VOTERS

FOR EACH ROW

BEGIN

UPDATE CONSTITUENCY

SET NOOFVOTERS = NOOFVOTERS + 1

WHERE CONSTID=:NEW.CONSTID;

END;

Trigger created.

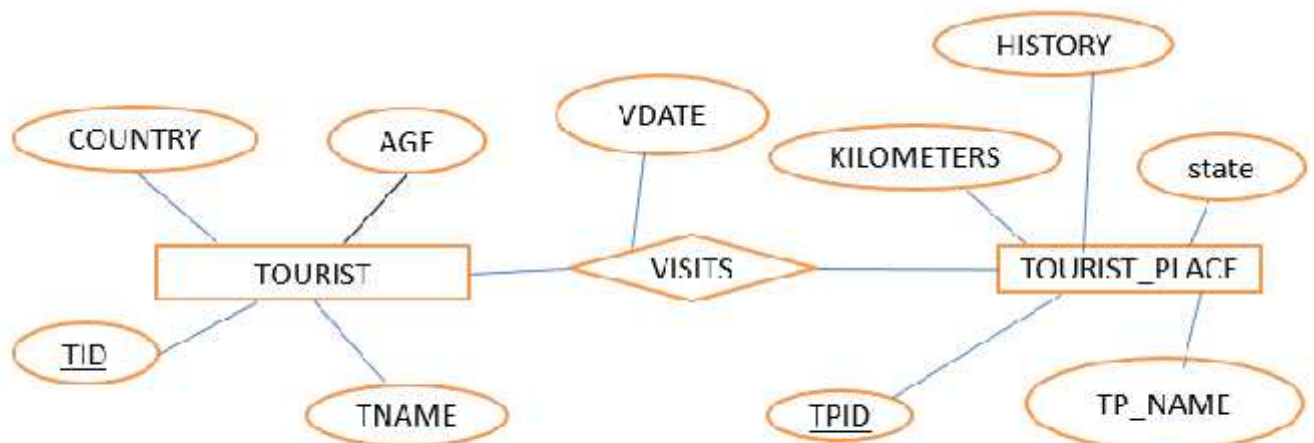
INSERT INTO VOTERS VALUES(10,'ARUN',22,'1 BALKI BIDAR KARNATAKA 577002',1,1);

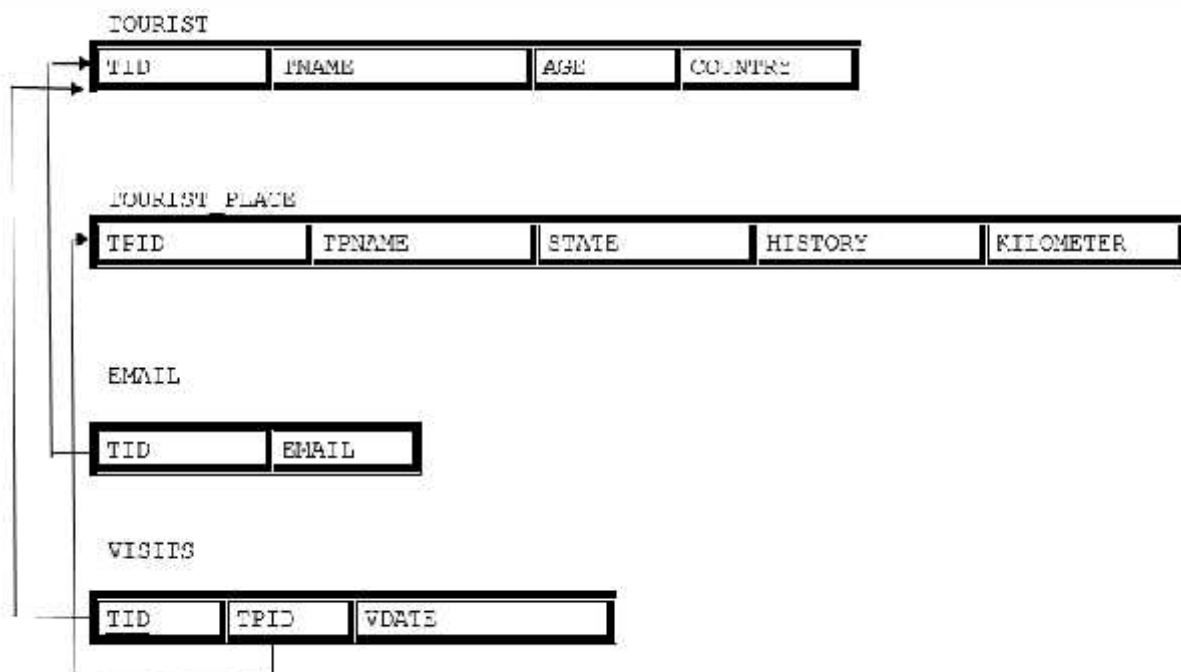
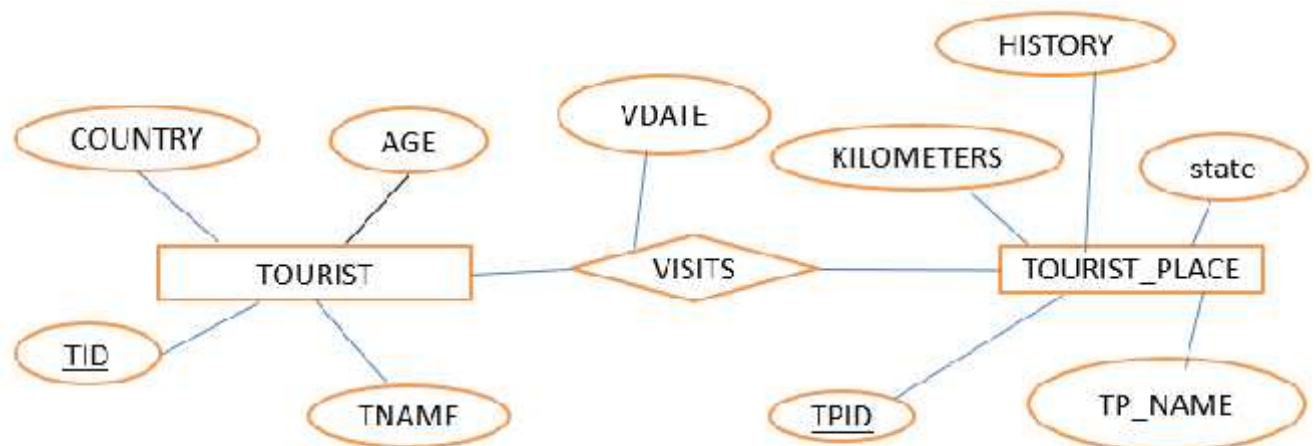
PROGRAM_5

Design an ER-diagram for the following scenario, convert the same into a relational model, normalize relations into a suitable Normal form and then solve the following queries. A country can have many Tourist places . Each Tourist place is identified by using tourist_place_id, having a name, belongs to a state, Number of kilometres away from the capital city of that state, history. There are many Tourists visits tourist places every year. Each tourist is identified uniquely by using Tourist_id, having a Name, age, Country and multiple emailids. A tourist visits many Tourist places, it is also required to record the visited_date in the database. A tourist can visit a Tourist place many times at different dates. A Tourist place can be visited by many tourists either in the same date or at different dates.

Queries:

1. List the state name which is having maximum number of tourist places.
2. List details of Tourist place where maximum number of tourists visited.
3. List the details of tourists visited all tourist places of the state "KARNATAKA".
4. Display the details of the tourists visited at least one tourist place of the state, but visited all states tourist places.
5. Display the details of the tourist place visited by the tourists of all country.





1. create table tourist_place (tpid number primary key, history varchar(20), kilometers number(3), state varchar(20), tpname varchar(20));
2. create table tourist(tid number primary key, country varchar(20), age number, tname varchar(20));
3. create table visits (tpid number(3) references tourist_place(tpid), tid number references tourist(tid), vdate date, primary key (tpid, tid));

4. create table email
 (tid number references tourist(tid),
 email varchar (20), primary key (tid, email));

```
desc tourist_place;
```

Name	Null?	Type
TPID	NOT NULL	NUMBER
HISTORY		VARCHAR2(20)
KILOMETERS		NUMBER
STATE		VARCHAR2(20)
TPNAME		VARCHAR2(20)

```
desc tourist;
```

Name	Null?	Type
TID	NOT NULL	NUMBER
COUNTRY		VARCHAR2(20)
NUMBER		
TNAME		VARCHAR2(20)

```
desc visits;
```

Name	Null?	Type
TPID	NOT NULL	NUMBER
TID	NOT NULL	NUMBER
VDATE		DATE

```
desc email;
```

Name	Null?	Type	
NUMBER			TID
EMAIL		VARCHAR2(20)	

```
SQL> insert into
tourist_place(tpid,history,kilometers,state,tpname)values(11,beauty,
160,'karnataka','ooty');
```

```
1 row created.
```

```
insert into tourist_place
values(tpid,history,kilometers,state,tpname)values('11','beauty',
'160','karnataka','ooty');
```

```
insert into tourist_place values(11,'beauty',160,'karnataka','ooty');
insert into tourist_place values(12,'monuments',270,'kerala','beluru');
insert into tourist_place values(13,'beach',360,'tamilnadu','marina');
```

```
SQL> select * from tourist_place;
```

TPID	HISTORY	KILOMETERS	STATE	TPNAME
11	beauty	160	karnataka	ooty
12	monuments	270	kerala	beluru
13	beach	360	tamilnadu	marina

```
insert into values(11,'Beauty',34,'prakash');
```

```
insert into tourist place values(12,'Monuments'270','prakash');
```

```
SQL> insert into
tourist(tid, country, age, tname) values(22, 'india', 34, 'prakash');
```

1 row created.

```
SQL> select * from tourist;
```

TID	COUNTRY	AGE	TNAME
22	india	34	prakash
23	orissa	28	bhanu
24	india	30	nagesh

```
SQL> insert into visits values('&tpid','&tid','&vdate');
```

Enter value for tpid: 12

Enter value for tid: 23

Enter value for vdate: 13-nov-2014

old 1: insert into visits values('&tpid','&tid','&vdate')

new 1: insert into visits values('12','23','13-nov-2014')

1 row created.

```
SQL> select * from visits;
```

TPID	TID	VDATE
12	23	13-NOV-14
11	24	24-JUN-13
13	22	25-SEP-11
11	23	23-FEB-10
13	23	12-JAN-10
14	24	10-JAN-17

```
SQL> insert into email values('&tid','&email');
```

Enter value for tid: 23

Enter value for email: bhanu12@gmail.com

old 1: insert into email values('&tid','&email')

new 1: insert into email values('23','bhanu12@gmail.com')

1 row created.

```
SQL> select * from email; TID
EMAIL
```

23	bhanu12@gmail.com
22	prakash242@gmail.com
24	nageshh@gmail.com

Queries:

1. List the state name which is having maximum number of tourist places.

Query 1:

```
select state from tourist_place group by state having count(state)=
(select max(count(state)) from tourist_place group by state);
```

STATE

karnataka

2. List details of Tourist place where maximum number of tourists visited.

query 2:

```
select * from tourist_place where tpid in (select tpid from visits group by
tpid having count(tpid)= (select max(count(tpid)) from visits group by
tpid));
```

TPID HISTORY	KILOMETERS STATE	TPNAME
11 beauty	160 karnataka	ooty
13 beach	360 tamilnadu	marina

3. List the details of tourists visited all tourist places of the state "KARNATAKA".

query 3:

```
select * from tourist t where t.tid in
(select tid from visits join tourist_place on
visits.tpid=tourist_place.tpid where state='karnataka'
group by tid having count(state) in (select count (state) from
tourist_place where state='karnataka'));
```

TID COUNTRY	AGE TNAME
24 india	30 nagesh

4. Display the details of the tourists visited at least one tourist place of the state, but visited all states tourist places.

query 4:

```
select * from tourist t where t.tid in (select tid from visits join
tourist_place on visits.tpid=tourist_place.tpid
group by tid having count(distinct state)
in (select count(distinct state ) from tourist_place) );
```

TID COUNTRY	AGE TNAME
23 orissa	28 bhanu

5. Display the details of the tourist place visited by the tourists of all country.

query 5:

```
select * from tourist_place where tpid in (
select tpid from visits join tourist on visits.tid=tourist.tid
group by tpid having count(distinct country)=
(select countdistinct country) from tourist));
```

TPID HISTORY	KILOMETERS STATE	TPNAME
11 beauty	160 karnataka	ooty

13 beach
Values(34,23,67)

360 tamilnadu

marina

Count()=3

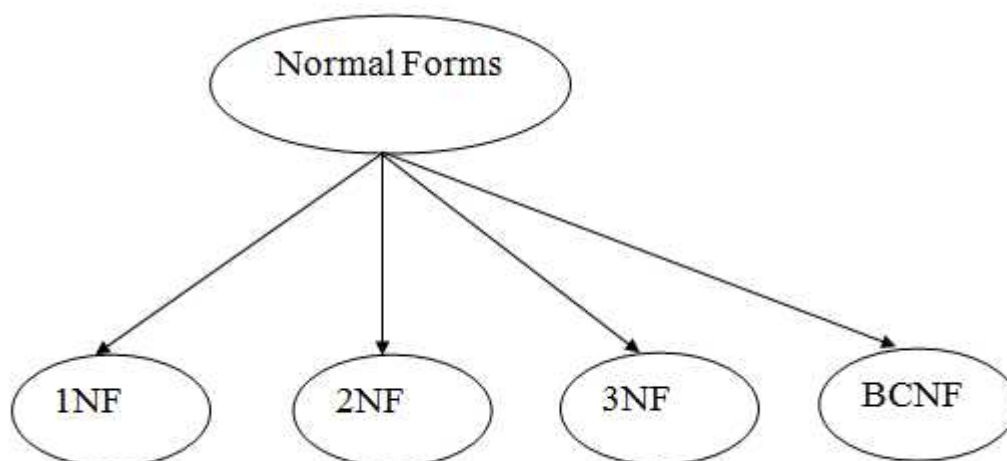
Count(max)

Normalization

- o Normalization is the process of organizing the data in the database.
- o Normalization is used to minimize the redundancy from a relation or set of relations. It is also used to eliminate the undesirable characteristics like Insertion, Update and Deletion Anomalies.
- o Normalization divides the larger table into the smaller table and links them using relationship.
- o The normal form is used to reduce redundancy from the database table.

Types of Normal Forms

There are the four types of normal forms:



Normal Form	Description
1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
3NF	A relation will be in 3NF if it is in 2NF and no transitive dependency exists.
4NF	A relation will be in 4NF if it is in Boyce Codd normal form and has no multi-valued dependency.
5NF	A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.

First Normal Form (1NF)

- o A relation will be 1NF if it contains an atomic value.
- o It states that an attribute of a table cannot hold multiple values. It must hold only single-valued attribute.
- o First normal form disallows the multi-valued attribute, composite attribute, and their combinations.

Example: Relation EMPLOYEE is not in 1NF because of multi-valued attribute EMP_PHONE.

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	7272826385, 9064738238	UP
20	Harry	8574783832	Bihar
12	Sam	7390372389, 8589830302	Punjab

The decomposition of the EMPLOYEE table into 1NF has been shown below:

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	7272826385	UP

14	John	9064738238	UP
20	Harry	8574783832	Bihar
12	Sam	7390372389	Punjab
12	Sam	8589830302	Punjab

Second Normal Form (2NF)

- o In the 2NF, relational must be in 1NF.
- o In the second normal form, all non-key attributes are fully functional dependent on the primary key

Example: Let's assume, a school can store the data of teachers and the subjects they teach. In a school, a teacher can teach more than one subject.

TEACHER table

TEACHER_ID	SUBJECT	TEACHER_AGE
25	Chemistry	30
25	Biology	30
47	English	35
83	Math	38
83	Computer	38

In the given table, non-prime attribute TEACHER_AGE is dependent on TEACHER_ID which is a proper subset of a candidate key. That's why it violates the rule for 2NF.

To convert the given table into 2NF, we decompose it into two tables:

TEACHER_DETAIL table:

TEACHER_ID	TEACHER_AGE
25	30

47	35
83	38

TEACHER_SUBJECT table:

TEACHER_ID	SUBJECT
25	Chemistry
25	Biology
47	English
83	Math
83	Computer

Third Normal Form (3NF)

- o A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.
- o 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- o If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

A relation is in third normal form if it holds atleast one of the following conditions for every non-trivial function dependency $X \rightarrow Y$.

1. X is a super key.
2. Y is a prime attribute, i.e., each element of Y is part of some candidate key.

Example:

EMPLOYEE_DETAIL table:

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harry	201010	UP	Noida
333	Stephan	02228	US	Boston

444	Lan	60007	US	Chicago
555	Katharine	06389	UK	Norwich
666	John	462007	MP	Bhopal

Super key in the table above:

1. {EMP_ID}, {EMP_ID, EMP_NAME}, {EMP_ID, EMP_NAME, EMP_ZIP}....so on
Candidate key: {EMP_ID}

Non-prime attributes: In the given table, all attributes except EMP_ID are non-prime.

Here, EMP_STATE & EMP_CITY dependent on EMP_ZIP and EMP_ZIP dependent on EMP_ID. The non-prime attributes (EMP_STATE, EMP_CITY) transitively dependent on super key(EMP_ID). It violates the rule of third normal form.

That's why we need to move the EMP_CITY and EMP_STATE to the new <EMPLOYEE_ZIP> table, with EMP_ZIP as a Primary key.

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_ZIP
222	Harry	201010
333	Stephan	02228
444	Lan	60007
555	Katharine	06389
666	John	462007

EMPLOYEE_ZIP table:

EMP_ZIP	EMP_STATE	EMP_CITY
201010	UP	Noida
02228	US	Boston
60007	US	Chicago
06389	UK	Norwich

462007

MP

Bhopal

Boyce Codd normal form (BCNF)

- o BCNF is the advance version of 3NF. It is stricter than 3NF.
- o A table is in BCNF if every functional dependency $X \rightarrow Y$, X is the super key of the table.
- o For BCNF, the table should be in 3NF, and for every FD, LHS is super key.

Example: Let's assume there is a company where employees work in more than one department.

EMPLOYEE table:

EMP_ID	EMP_COUNTRY	EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
264	India	Designing	D394	283
264	India	Testing	D394	300
364	UK	Stores	D283	232
364	UK	Developing	D283	549

In the above table Functional dependencies are as follows:

1. $EMP_ID \rightarrow EMP_COUNTRY$
2. $EMP_DEPT \rightarrow \{DEPT_TYPE, EMP_DEPT_NO\}$

Candidate key: {EMP-ID, EMP-DEPT}

The table is not in BCNF because neither EMP_DEPT nor EMP_ID alone are keys.

To convert the given table into BCNF, we decompose it into three tables:

EMP_COUNTRY table:

EMP_ID	EMP_COUNTRY
264	India
264	India

EMP_DEPT table:

EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
Designing	D394	283

Testing	D394	300
Stores	D283	232
Developing	D283	549

EMP_DEPT_MAPPING table:

EMP_ID	EMP_DEPT
D394	283
D394	300
D283	232
D283	549

Functional dependencies:

1. EMP_ID → EMP_COUNTRY
2. EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

Candidate keys:

For the first table: EMP_ID

For the second table: EMP_DEPT

For the third table: {EMP_ID, EMP_DEPT}

Now, this is in BCNF because left side part of both the functional dependencies is a key.

SQL Command to Practice

SQL Statement

• **Syntax**

AND / OR

• SELECT column_name(s)
FROM table_name
WHERE condition
AND/OR condition

ALTER TABLE	● ALTER TABLE table_name ADD column_name datatype ● or ● ALTER TABLE table_name DROP COLUMN column_name
AS (alias)	● SELECT column_name AS column_alias FROM table_name ● or ● SELECT column_name FROM table_name AS table_alias
BETWEEN	● SELECT column_name(s) FROM table_name WHERE column_name BETWEEN value1 AND value2
CREATE DATABASE	● CREATE DATABASE database_name
CREATE TABLE	● CREATE TABLE table_name (column_name1 data_type, column_name2 data_type, column_name3 data_type, ...)
CREATE INDEX	● CREATE INDEX index_name ON table_name (column_name) ● or ● CREATE UNIQUE INDEX index_name ON table_name (column_name)
CREATE VIEW	● CREATE VIEW view_name AS SELECT column_name(s) FROM table_name WHERE condition
DELETE	● DELETE FROM table_name WHERE some_column=some_value ● or ● DELETE FROM table_name (Note: Deletes the entire table!!) ● DELETE * FROM table_name (Note: Deletes the entire table!!)

DROP DATABASE	● DROP DATABASE database_name
DROP INDEX	● DROP INDEX table_name.index_name (SQL Server) ● DROP INDEX index_name ON table_name (MS Access) ● DROP INDEX index_name (DB2/Oracle) ● ALTER TABLE table_name ● DROP INDEX index_name (MySQL)
DROP TABLE	● DROP TABLE table_name
EXISTS	● IF EXISTS (SELECT * FROM table_name WHERE id = ?) ● BEGIN ● --do what needs to be done if exists ● END ● ELSE ● BEGIN ● --do what needs to be done if not ● END
GROUP BY	● SELECT column_name, aggregate_function(column_name) ● FROM table_name ● WHERE column_name operator value ● GROUP BY column_name
HAVING	● SELECT column_name, aggregate_function(column_name) ● FROM table_name ● WHERE column_name operator value ● GROUP BY column_name ● HAVING aggregate_function(column_name) operator value
IN	● SELECT column_name(s) ● FROM table_name ● WHERE column_name ● IN (value1,value2,..)
INSERT INTO	● INSERT INTO table_name ● VALUES (value1, value2, value3,...) ● or ● INSERT INTO table_name ● (column1, column2, column3,...) ● VALUES (value1, value2, value3,...)
INNER JOIN	● SELECT column_name(s) ● FROM table_name1 ● INNER JOIN table_name2 ● ON table_name1.column_name=table_name2.column_name
LEFT JOIN	● SELECT column_name(s) ● FROM table_name1

	LEFT JOIN table_name2 ON table_name1.column_name=table_name2.column_name
RIGHT JOIN	SELECT column_name(s) FROM table_name1 RIGHT JOIN table_name2 ON table_name1.column_name=table_name2.column_name
FULL JOIN	SELECT column_name(s) FROM table_name1 FULL JOIN table_name2 ON table_name1.column_name=table_name2.column_name
LIKE	SELECT column_name(s) FROM table_name WHERE column_name LIKE pattern
ORDER BY	SELECT column_name(s) FROM table_name ORDER BY column_name [ASC DESC]
SELECT	SELECT column_name(s) FROM table_name
SELECT *	SELECT * FROM table_name
SELECT DISTINCT	SELECT DISTINCT column_name(s) FROM table_name
SELECT INTO	- SELECT * INTO new_table_name [IN externaldatabase] FROM old_table_name - or - SELECT column_name(s) INTO new_table_name [IN externaldatabase] FROM old_table_name
SELECT TOP	SELECT TOP number percent column_name(s) FROM table_name
TRUNCATE TABLE	TRUNCATE TABLE table_name
UNION	SELECT column_name(s) FROM table_name1 UNION SELECT column_name(s) FROM table_name2
UNION ALL	SELECT column_name(s) FROM table_name1 UNION ALL SELECT column_name(s) FROM table_name2

UPDATE

- UPDATE table_name
SET column1=value, column2=value,...
WHERE some_column=some_value

WHERE

- SELECT column_name(s)
FROM table_name
WHERE column_name operator value